

Modes de Fonctionnement

Docker par Type de Projet

Fichiers à créer · Exemples concrets · Bonnes pratiques

Dev

Prod

Micro-services

CI/CD

Pipeline



Modes de Fonctionnement

Docker par Type de Projet

Fichiers à créer · Exemples concrets · Bonnes pratiques

Dev

Prod

Micro-services

CI/CD

Pipeline



01 Introduction à Docker

Concepts fondamentaux

Docker est une plateforme de **conteneurisation** qui permet d'empaqueter une application et toutes ses dépendances dans un **conteneur isolé**. Contrairement aux machines virtuelles, les conteneurs partagent le noyau de l'OS hôte, ce qui les rend ultra-légers et rapides.

Concept	Description	Analogie
Image	Blueprint immuable de l'application	Recette de cuisine
Conteneur	Instance en cours d'exécution d'une image	Plat cuisiné
Dockerfile	Script de construction de l'image	Recette écrite
docker-compose	Orchestration multi-conteneurs	Menu complet
Volume	Stockage persistant monté dans un conteneur	Réfrigérateur partagé
Network	Réseau virtuel entre conteneurs	Table commune

Pourquoi Docker ?

- Environnements reproductibles : "ça marche sur ma machine" disparaît
- Isolation : chaque service tourne dans son propre sandbox
- Scalabilité : facile de dupliquer ou déployer des conteneurs
- Portabilité : fonctionne identiquement en local, staging et production
- Écosystème riche : Docker Hub, Compose, Swarm, Kubernetes

■ ■ Vue d'ensemble des modes

Docker s'adapte à **5 types de projets principaux**, chacun avec ses propres besoins en termes de fichiers, de configuration et d'architecture :

Mode	Type de projet	Complexité	Fichiers clés
■ ■ Dev	Projet solo / apprentissage	■	Dockerfile

Mode	Type de projet	Complexité	Fichiers clés
■ Web Full-Stack	App web avec BDD	■ ■	Dockerfile + compose.yml
■ Micro-services	Architecture distribuée	■ ■ ■	Multiple Dockerfiles + compose
■ Production	Déploiement en ligne	■ ■ ■ ■	Dockerfile multi-stage + secrets
■ ■ CI/CD	Pipeline automatisé	■ ■ ■ ■ ■	Dockerfile + .github/workflows

02 Mode Développement

Projet solo · Apprentissage · Prototype

Le mode développement est idéal pour tester une technologie, développer une petite application ou isoler son environnement de travail. On monte le code source en **volume** pour éviter de rebuild l'image à chaque changement.

Caractéristiques

- Un seul conteneur (pas de compose obligatoire)
- Hot-reload grâce aux volumes bind-mount
- Port exposé pour accéder depuis le navigateur
- Variables d'environnement en clair (pas de secrets)

■ Fichiers à créer

■ Dockerfile

```
# Dockerfile – Application Node.js en développement
FROM node:20-alpine

# Dossier de travail dans le conteneur
WORKDIR /app

# Copier et installer les dépendances
COPY package*.json ./
RUN npm install

# Copier le code source
COPY . .

# Port exposé
EXPOSE 3000

# Commande de démarrage
CMD ["npm", "run", "dev"]
```

Image Node.js minimaliste basée sur Alpine Linux (légère ~50MB)

■ .dockerignore

```
# .dockerignore – Exclure les fichiers inutiles
node_modules
.git
.env
*.log
dist/
coverage/
```

Évite de copier node_modules (lourd) et les secrets dans l'image

■ ■ Commandes essentielles

■ Terminal — Commandes de base

```
# Construire l'image
docker build -t mon-app:dev .

# Lancer avec hot-reload (code monté en volume)
docker run -p 3000:3000 -v $(pwd):/app mon-app:dev

# Voir les conteneurs actifs
docker ps

# Entrer dans un conteneur
docker exec -it <id> sh

# Arrêter et supprimer
docker stop <id> && docker rm <id>
```

03 Mode Web Full-Stack

App + Base de données · Docker Compose

La majorité des projets web nécessitent au minimum une application et une base de données. **Docker Compose** permet de définir et lancer plusieurs services en une seule commande. Chaque service est isolé mais peut communiquer via un réseau Docker interne.

Architecture typique

- Service frontend : React / Vue / Angular (port 3000)
- Service backend : Node.js / Python / Java (port 8000)
- Base de données : PostgreSQL / MySQL / MongoDB (port 5432)
- Cache optionnel : Redis (port 6379)
- Reverse proxy optionnel : Nginx (port 80)

■ Structure du projet

■ Structure des dossiers

```
mon-projet/  
├── backend/  
│   ├── Dockerfile  
│   ├── requirements.txt (ou package.json)  
│   └── app/  
├── frontend/  
│   ├── Dockerfile  
│   └── src/  
├── docker-compose.yml ← fichier central  
├── docker-compose.dev.yml ← surcharge dev  
├── .env ← variables d'environnement  
└── .dockerignore
```

■ docker-compose.yml — Exemple complet Python + PostgreSQL

```
version: '3.9'  
  
services:  
  # ■ Backend Python/FastAPI  
  backend:  
    build: ./backend  
    ports:  
      - "8000:8000"  
    environment:  
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb  
    volumes:  
      - ./backend:/app # hot-reload  
    depends_on:  
      db:  
        condition: service_healthy
```



```
docker compose up -d --build

# Arrêter sans supprimer les volumes
docker compose down

# Arrêter ET supprimer les données
docker compose down -v
```

04 Mode Micro-services

Architecture distribuée · Services indépendants

04 Mode Micro-services

Architecture distribuée · Services indépendants

Dans une architecture micro-services, chaque fonctionnalité est encapsulée dans un service indépendant avec son propre Dockerfile, sa propre base de données et son cycle de déploiement. Docker Compose orchestre l'ensemble en local ; **Kubernetes** prend le relais en production.

Principes clés

- Un service = un conteneur = une responsabilité unique
- Communication via API REST ou message broker (RabbitMQ, Kafka)
- Chaque service a sa propre BDD (pas de BDD partagée)
- Un API Gateway centralise les requêtes entrantes
- Health checks obligatoires pour la résilience

- ## Principes clés
- Un service = un conteneur = une responsabilité unique
 - Communication via API REST ou message broker (RabbitMQ, Kafka)
 - Chaque service a sa propre BDD (pas de BDD partagée)
 - Un API Gateway centralise les requêtes entrantes
 - Health checks obligatoires pour la résilience

- Structure micro-services

```
■ Structure micro-services
ecommerce/
  ■■■ services/
    ■ ■■■ api-gateway/
      ■ ■ ■ Dockerfile
      ■ ■ ■ nginx.conf
    ■■■ user-service/
      ■ ■■■ Dockerfile
      ■ ■ ■ src/
    ■■■ product-service/
      ■ ■■■ Dockerfile
      ■ ■ ■ src/
    ■■■ order-service/
      ■ ■■■ Dockerfile
      ■ ■ ■ src/
  ■■■ docker-compose.yml
  ■■■ .env
```

■ docker-compose.yml — Architecture micro-services e-commerce

[illegible]

05 Mode Production

Multi-stage build · Sécurité · Optimisation

05 Mode Production

Multi-stage build · Sécurité · Optimisation

En production, la priorité est la **sécurité**, la **performance** et la **taille de l'image**. Le *multi-stage build* permet de compiler l'application dans un environnement complet, puis de ne copier que les artéfacts nécessaires dans une image finale ultra-légère.

Critère	Dev	Production
Taille image	~500 MB (outils de dev inclus)	~50 MB (multi-stage)
Secrets	Variables en clair dans .env	Docker secrets / Vault
User	root (par défaut)	Utilisateur non-privilégié
Volumes	Code monté (hot-reload)	Données uniquement
Debug	Activé	Désactivé
Logs	Console	Système de log centralisé

■ Dockerfile — Multi-stage build (Node.js)

```
# ■■■ STAGE 1 : Build ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  
FROM node:20-alpine AS builder  
  
WORKDIR /app  
COPY package*.json ./  
RUN npm ci --only=production  
COPY . .  
RUN npm run build  
  
# ■■■ STAGE 2 : Production (image finale légère) ■■■■  
FROM node:20-alpine AS production  
  
# Créer un utilisateur non-root (sécurité)  
RUN addgroup -S appgroup && adduser -S appuser -G appgroup  
  
WORKDIR /app  
  
# Copier UNIQUEMENT ce qui est nécessaire  
COPY --from=builder /app/node_modules ./node_modules  
COPY --from=builder /app/dist ./dist  
COPY --from=builder /app/package.json .  
  
# Passer à l'utilisateur non-privilégié  
USER appuser  
  
# Métadonnées de l'image  
LABEL maintainer="team@monapp.com"
```

```
LABEL version="1.0.0"

EXPOSE 3000

CMD ["node", "dist/index.js"]
```

L'image finale ne contient pas les outils de build, les tests, ou le code source brut.

■ docker-compose.prod.yml — Configuration production

```
version: '3.9'

services:
  app:
    image: mon-app:${APP_VERSION}
    restart: always
    deploy:
      replicas: 3
      resources:
        limits:
          cpus: '0.5'
          memory: 512M
    secrets:
      - db_password
      - jwt_secret
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

secrets:
  db_password:
    external: true
  jwt_secret:
    external: true
```

Les secrets ne sont jamais dans les variables d'environnement en production.

06 Mode CI/CD

Intégration continue · Déploiement automatisé

Le pipeline CI/CD automatise le test, la construction et le déploiement de l'application. Docker garantit que l'environnement est identique à chaque étape : du poste du développeur jusqu'au serveur de production.

Pipeline typique

- 1. Push du code → déclenche le pipeline
- 2. Build de l'image Docker (multi-stage)
- 3. Exécution des tests dans le conteneur
- 4. Push de l'image sur le registry (Docker Hub / ECR / GCR)
- 5. Déploiement automatique sur le serveur

■ .github/workflows/docker.yml — GitHub Actions

```
name: Docker CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Log in to Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${ env.REGISTRY }/${ env.IMAGE_NAME }:latest
```

```

cache-from: type=gha
cache-to: type=gha,mode=max

deploy:
  needs: build-and-push
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Deploy via SSH
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.SERVER_HOST }
        username: ${ secrets.SERVER_USER }
        key: ${ secrets.SSH_PRIVATE_KEY }
        script: |
          docker pull ghcr.io/${ github.repository }:latest
          docker compose -f /app/docker-compose.prod.yml up -d

```

Le cache GitHub Actions accélère les builds en réutilisant les layers Docker.

■ Makefile — Commandes courantes (optionnel mais pratique)

```

# Makefile
.PHONY: dev build test deploy

dev:      ## Lancer en développement
docker compose up -d

build:    ## Construire l'image de prod
docker build -t mon-app:latest --target production .

test:     ## Lancer les tests dans Docker
docker run --rm mon-app:latest npm test

push:     ## Pousser sur le registry
docker push mon-app:latest

clean:    ## Nettoyer les ressources Docker
docker system prune -f

```

07 Récapitulatif & Bonnes Pratiques

Checklist · Tips · Pièges à éviter

■ Checklist par type de projet

Fichier	Dev	Full-Stack	Micro-svcs	Prod	CI/CD
Dockerfile	■	■	■ (x N)	■ multi-stage	■
.dockerignore	■	■	■	■	■
docker-compose.yml	■	■	■	■	■
.env	■	■	■	■ (secrets)	■ (GitHub secrets)
nginx.conf	■	Optionnel	■ (gateway)	■	■
.github/workflows/	■	■	Optionnel	■	■
Makefile	Optionnel	Optionnel	■	■	Optionnel

■ Pièges fréquents à éviter

- **Ne jamais committer .env** — Ajouter dans .gitignore, utiliser .env.example
- **Ne pas lancer en root en prod** — Créer un utilisateur dédié dans le Dockerfile
- **Ordre des COPY dans Dockerfile** — Copier package.json avant le code source pour profiter du cache
- **Pas de tag 'latest' en prod** — Toujours utiliser des tags de version (1.2.3, commit SHA)
- **Oublier les health checks** — Compose attend le 'healthy' avant de démarrer les dépendants
- **Exposer les ports de BDD** — La DB ne doit être accessible que depuis le réseau interne Docker
- **Images trop volumineuses** — Utiliser alpine, multi-stage, et optimiser les layers

■ Tips de performance

- Placer les instructions qui changent rarement en premier (COPY package.json avant COPY .)
- Utiliser 'docker compose watch' pour le hot-reload sans volumes complexes
- Activer BuildKit : DOCKER_BUILDKIT=1 docker build ... (plus rapide)
- Utiliser 'docker system df' pour voir l'espace utilisé par Docker

■ Nettoyer régulièrement : `docker system prune --volumes`

■ Ressources

- Documentation officielle : docs.docker.com
- Docker Hub (images officielles) : hub.docker.com
- Play with Docker (sandbox en ligne) : labs.play-with-docker.com
- Dive (analyser les layers d'une image) : github.com/wagoodman/dive

Briefing Docker — Tous droits réservés · Généré avec ReportLab